# Internalizing Safety: Control Barrier Functions for Socially Aware Multi-Agent Reinforcement Learning

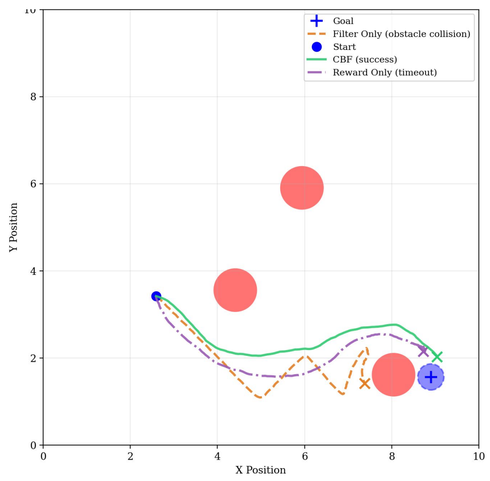
**Presenters**: Hisham Bhatti and Joshua Ng

**CSE 579 – Spring 2026**

## Motivation

- RL can often prioritize performance at the expense of safety → catastrophic in real-world
- Integrate safety mechanisms (CBFs) into training via *safety filters*
  1. Project RL action onto safe set before execution (filter)
  2. Augments reward term to penalize unsafe states (reward)

Yang et al. [1] proposes a dual approach: **CBF-RL**




## Background

### CBF Safety Filter [1] [2]:

- Define safe set C = {**x** | h(**x**) ≥ 0}
- **Control Barrier Function (CBF)** enforces that the robot will not leave the safe set (i.e. forward invariance of the safe set C).

| | Single Integrator ($\dot{x} = \mathbf{u}$) | Double Integrator ($\dot{x} = \mathbf{v}$, $\dot{v} = \mathbf{a}$) |
|---|---|---|
| Safety condition | $\nabla h \cdot \mathbf{u} + \alpha h \geq 0$ | $\nabla h \cdot \mathbf{a} + \mathbf{v}^\top \nabla^2 h\, \mathbf{v} + 2(\nabla h \cdot \mathbf{v}) + h \geq 0$ |
| Violation measure | $\psi = \nabla h \cdot \mathbf{u}^* + \alpha h$ | $\text{slack} = \nabla h \cdot \mathbf{a}^* + \mathbf{v}^\top \nabla^2 h\, \mathbf{v} + 2(\nabla h \cdot \mathbf{v}) + h$ |
| Correction (if violated) | $\mathbf{u}^{\text{safe}} = \mathbf{u}^* - \frac{\psi}{\lvert\nabla h\rvert^2} \nabla h$ | $\mathbf{a}^{\text{safe}} = \mathbf{a}^* - \frac{\text{slack}}{\lvert\nabla h\rvert^2} \nabla h$ |

### Reward Terms [1]:

| Reward | Formula |
|---|---|
| $r_{\text{goal}}$ | $1.0 \cdot \mathbf{1}(\text{goal reached})$ |
| $r_{\text{obstacle}}$ | $-1.0 \cdot \mathbf{1}(\text{obstacle collision})$ |
| $r_{\text{wall}}$ | $-1.0 \cdot \mathbf{1}(\text{wall collision})$ |
| $r_{\text{progress}}$ | $20.0 \cdot \frac{\lVert\mathbf{p}_{t-1} - \mathbf{g}\rVert - \lVert\mathbf{p}_t - \mathbf{g}\rVert}{v_{\max}\, \Delta t} \cdot \mathbf{1}(\text{active})$ |
| $r_{\text{alive}}$ | $0.01 \cdot \mathbf{1}(\text{active})$ |
| $r_{\text{cbf}}$ | $100 \cdot \Big(\min(\nabla h(\mathbf{q})^\top \mathbf{v} + \alpha h(\mathbf{q}),\, 0) + \exp\Big(-\frac{\lVert\mathbf{v}_{\text{policy}} - \mathbf{v}_{\text{safe}}\rVert^2}{0.5^2}\Big) - 1\Big) \cdot \mathbf{1}(\text{active})$ |
| $r_{\text{timeout}}$ | $-10.0 \cdot \mathbf{1}(\text{time exceeded})$ |

## Problem

**Can this CBF-RL framework be extended in a multi-agent setting with a double-integrator dynamic model and stochastic control noise?**

## Reflection

- Dynamics did not affect success rate, but fewer timeouts
- Control noise had little effect
- All ablations performed worse in multi-agent, esp. CBF (deadlocks)

**Conclusion**: Naive CBF fails to generalize to more sophisticated scenarios

## Methods

### Dynamics Models:

| Model | State | Action | Equation |
|---|---|---|---|
| Single-Integrator | Position $\mathbf{x}$ | Velocity $\mathbf{u}$ | $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{u}\,\Delta t$ |
| Double-Integrator | Position + Velocity $(\mathbf{x}, \mathbf{v})$ | Acceleration $\mathbf{a}$ | $\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{a}\,\Delta t$, $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}\,\Delta t$ |

### Multi-Agent Extension:

- Each agent treats other agents as dynamic circular obstacles with radius $r_{\text{robot}}$
- Decentralized: each agent solves its own QP independently at every timestep
- Other agents' current positions are read at the start of each timestep (simultaneous parallel update) [3]

### Stochastic Control Noise:

Gaussian noise applied after CBF filter

$$\mathbf{u}_{\text{exec}} = \mathbf{u}_{\text{safe}} + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}\big(0,\, (\sigma_{\text{level}} \cdot u_{\max})^2 I\big)$$

| Level | $\sigma$ (fraction of $u_{\max}$) |
|---|---|
| Low | 0.1 |
| Medium | 0.3 |
| High | 0.5 |

### Ablation:

| | No CBF Reward | CBF Reward |
|---|---|---|
| **No CBF Filter** | Naive | Reward Only |
| **CBF Filter** | Filter Only | CBF (Full) |

## Experiments

### Experiment Grid:

| Suite | Sweep Variable | Values |
|---|---|---|
| 1 (8 runs) | Dynamics × Method | {single-integrator, double-integrator} × {naive, reward-only, filter-only, cbf} |
| 2 (60 runs) | Agents/Obstacles ($A + O = 5$) | $(1,4),\ (2,3),\ (3,2),\ (4,1),\ (5,0)$ |
| | Noise Level | low, medium, high |
| | Method | naive, reward-only, filter-only, cbf |

### Training Setup [1]:

- PPO via rsl-rl, 4096 parallel worlds, 1500 iterations
- MLP policy: obs_dim → 32 → 32 → 2 (actor), obs_dim → 32 → 32 → 1 (critic), ReLU
- $\gamma = 0.99$, $\lambda = 0.95$, clip $\varepsilon = 0.2$, lr $= 3 \times 10^{-4}$
- Evaluated over 1000 test episodes per trained run

## References

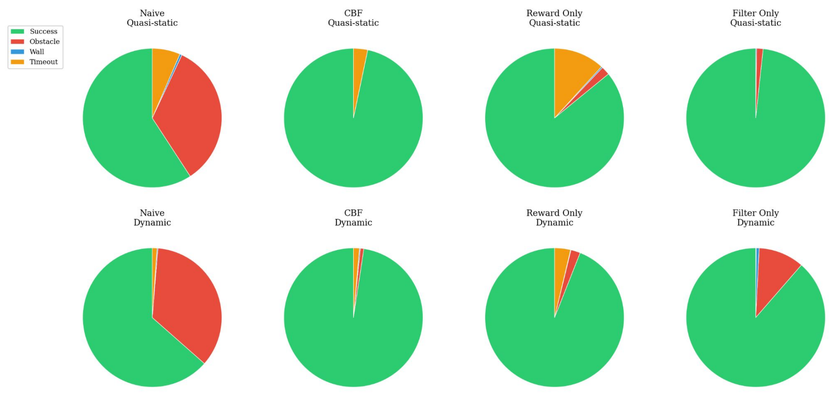
[1] Luo Yang, Bryan Werner, Mateus de Sa, and Aaron D. Ames. Cbf-rl: Safety filtering reinforcement learning in training with control barrier functions. arXiv preprint arXiv:2510.14959, 2025.

[2] Quan Nguyen and Koushil Sreenath. Exponential control barrier functions for enforcing high relative-degree safety-critical constraints. In American Control Conference (ACC), pp. 322–328, 2016.

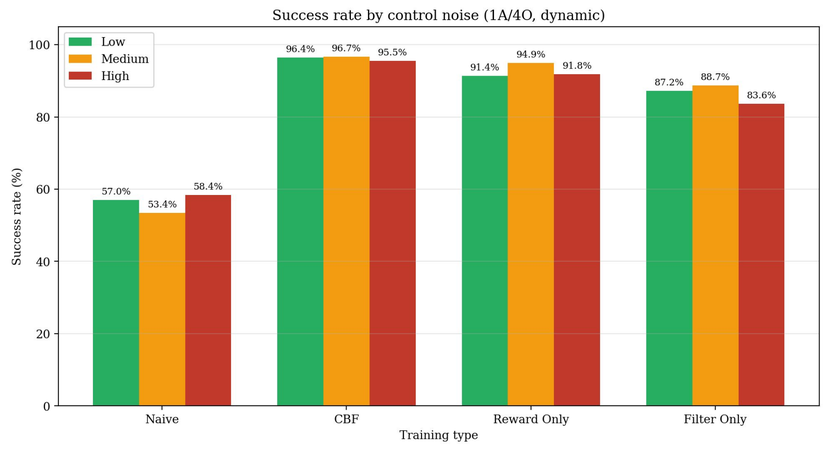
[3] Liqian Wang, Aaron D. Ames, and Magnus Egerstedt. Safety barrier certificates for collisions-free multirobot systems. IEEE Transactions on Robotics, 33(3):661–674, 2017.

## Results

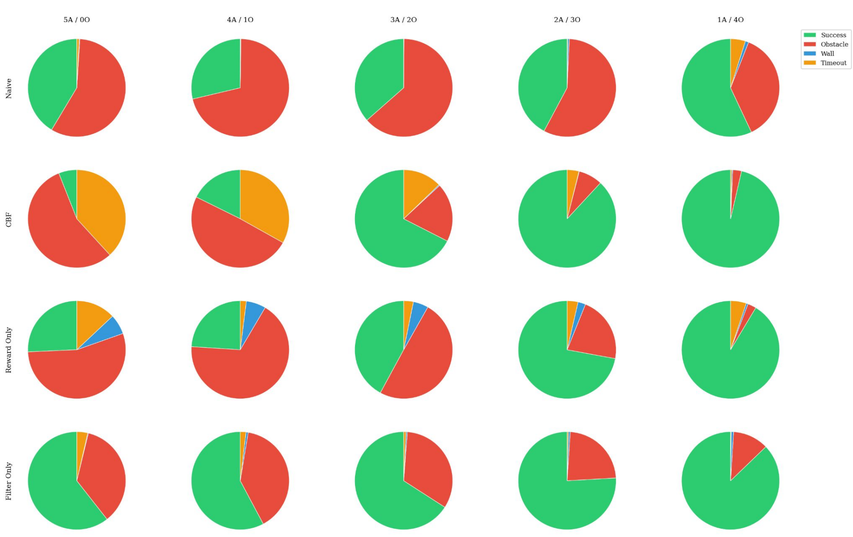
### Quasi-Static vs. Dynamic (1A, 3O):



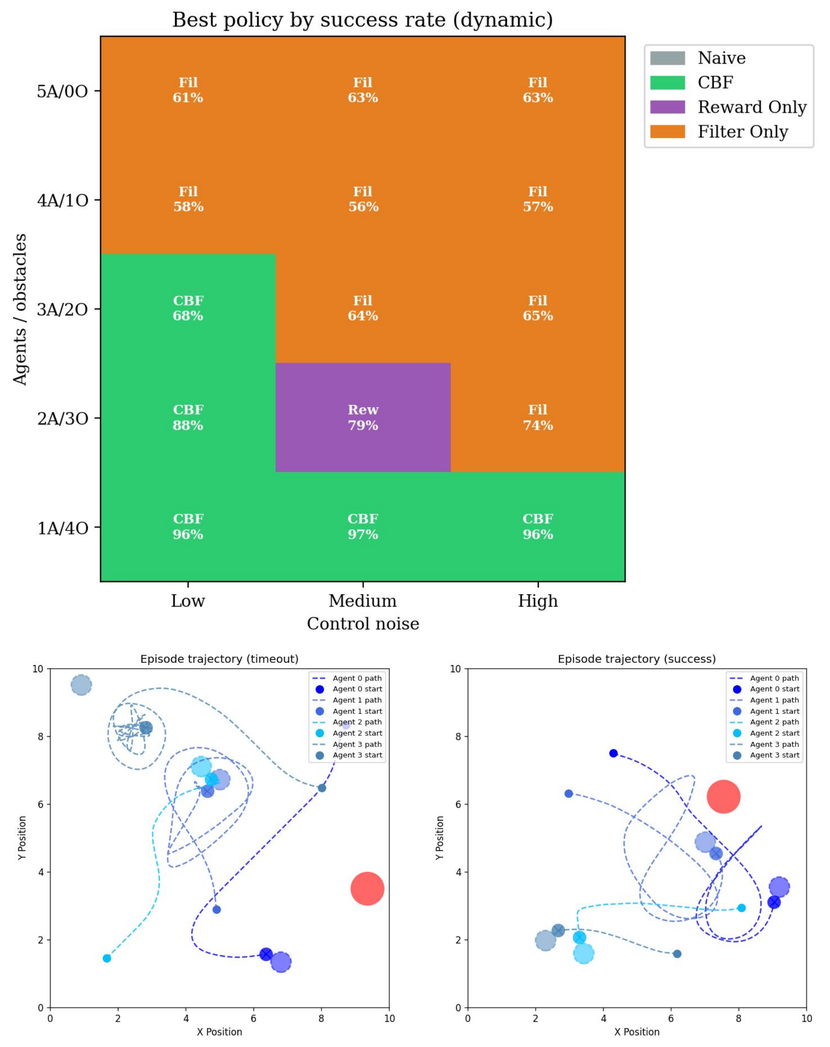

### Success Rate vs. Control Noise (1A, 4O):




### Success Rate vs. Agent/Obstacle Mix:




### Best Policy per Ablation:




**CBF (4A, 1O)**

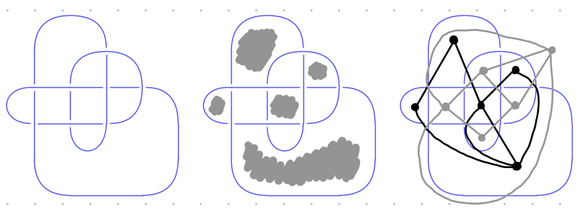
# An Algorithm for Implementing Tait Graph Functionality in SageMath

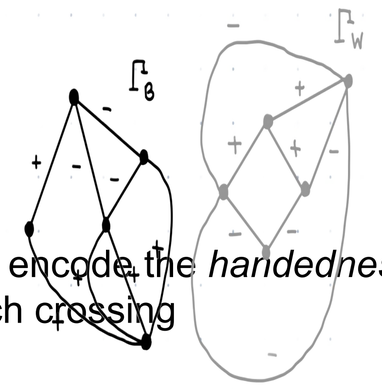
**Presenter: Hisham Bhatti, California State University: Chico Mathematics REU 2024**

**Mentor: Dr. John Lind**

## BACKGROUND

- A **knot** K is a smooth embedding of the circle $S_1$ onto $\mathbb{R}^3$
- We look at a **knot diagram** D, a projection of K onto $\mathbb{R}^2$
- The **Tait Graph** Γ is an undirected, +/- weighted graph associated to a knot diagram
  - Two disjoint Tait graphs: $\Gamma_B$ and $\Gamma_W$






- We encode the *handedness* of each crossing

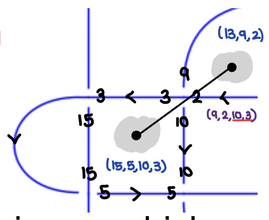

Left over Right

Right over Left

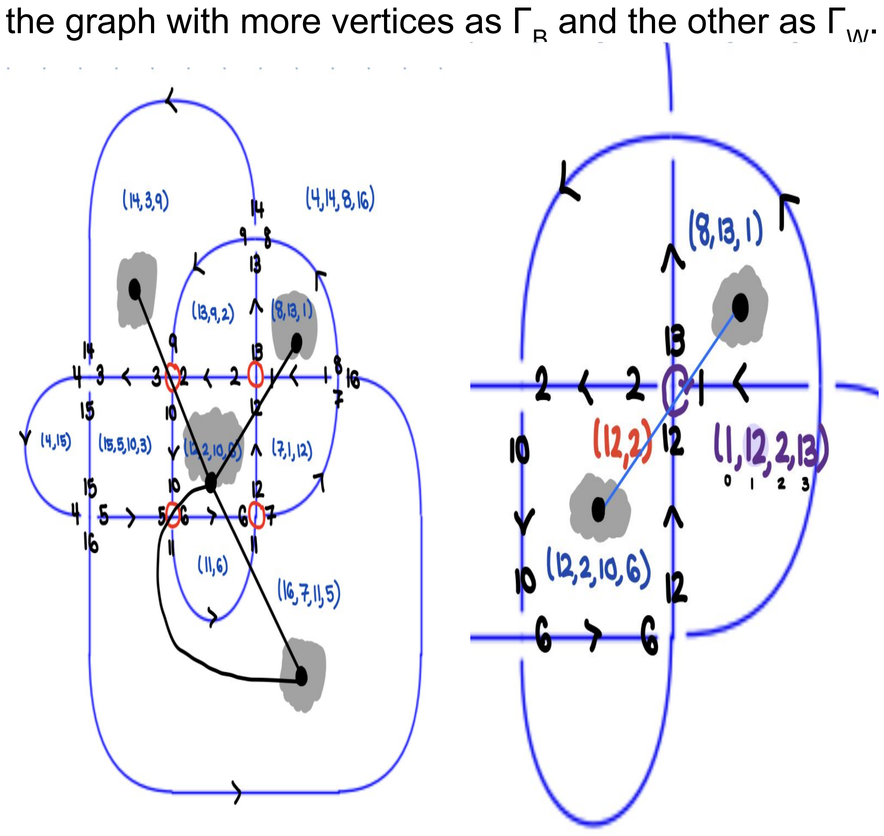
## RESEARCH OBJECTIVE

**Design an algorithm to construct a knot diagram's $\Gamma_B$ and $\Gamma_W$ Tait graph given its PD-code.**

## Regions Algorithm




1. Index the regions arbitrarily (1, 2, 3, …
2. Start with the first region $R_1$. Find crossings *c* which share two elements with the region. Such crossings are the *corners* of $R_1$.
1. Find the other *corner* in *c*. This corner is in a diagonal region $R_k$.
2. Continue this process starting at $R_2$ up to $R_n$, finding diagonal regions. Each region corresponds to a vertex, and crossings are edges between them.
3. Split the graph into its disconnected components. Return the graph with more vertices as $\Gamma_B$ and the other as $\Gamma_W$.

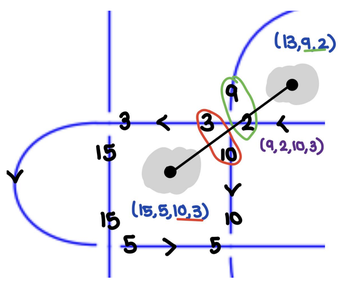



## Crossings Algorithm

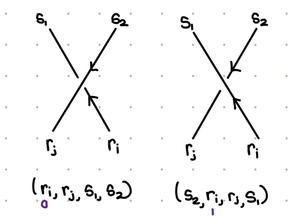
Given a region $R = (r_1, r_2, \ldots, r_k)$, a neighbor $S = (s_1, s_2, \ldots, s_l)$, and a crossing *C*

1. Decompose region *R* into cyclic pairs $[(r_1, r_2), (r_2, r_3), \ldots, (r_k, r_1)]$. Such pairs are the *corners* of *R*.
2. Find which pair $(r_i, r_j)$ matches the two strands at the crossing.

Cyclic Pairs for Region [15, 5, 10, 3]:




[15, 5] [5, 10] [**10, 3**] [3, 15]

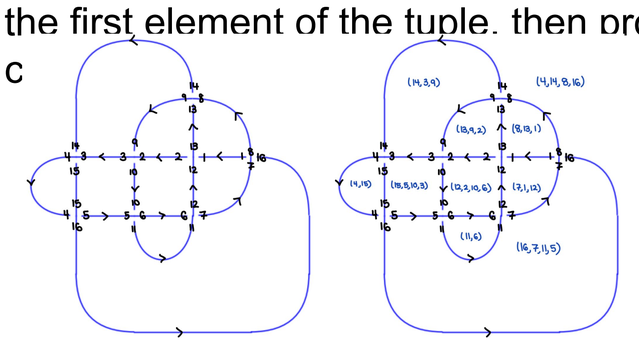



Note: A crossing in a PD code is the following: $C = (\underbrace{a}_{U}, \underbrace{b}_{O}, \underbrace{c}_{U}, \underbrace{d}_{O})$

3. Ind $C(r_i) = 0$ or $2 \rightarrow$ left over right (+) Ind $C(r_i) = 1$ or $3 \rightarrow$ right over left (-1)

## Planar Diagram (PD) Code

- Label an arbitrary incoming understrand with 1
- Follow along the knot with the orientation. When we hit a crossing, increment the label of the following strand by 1
- The incoming understrand will always be the first element of the tuple, then proceed c




(1, 12, 2, 13) (9, 2, 10, 3) (14, 3, 15, 4) (4, 15, 5, 16)
(10, 6, 11, 5) (6, 12, 7, 11) (16, 7, 1, 8) (13, 9, 14, 8)

## Regions

- Each region is defined by a tuple of the labels of the arcs enclosing it

## Results

- Tested our algorithm on 3000 knots up to 12 crossings (see KnotInfo)
  - Given a graph **Γ** = (V, E) with |V| = n, let $\mathbb{R}^{nxn}$,

$$D_{ii} = \sum_{e=(i,j)\in E} w_e \qquad A_{ij} = \sum_{e=(i,j)\in E} w_e$$

where $w_e$ = +1 if *e* is decorated with (+), $w_e$ = -1 if *e* is decorated with (-)

  - Define $\mathcal{L}$ = **D** - **A**. Construct the **Reduced Weighted Laplacian** $\mathcal{L}'$ by removing any row and column of $\mathcal{L}$
  - Should be that det($\mathcal{L}'$) = det(**G**), where **G** is the *Goertz Matrix*

See code at tinyurl.com/tait-graph

## Future Work

- Provably distinguish between $\Gamma_B$ and $\Gamma_W$
- Handle $R_1$ moves
- Calculate Jones Polynomial
- Implement Silver-Williams Directed Edges

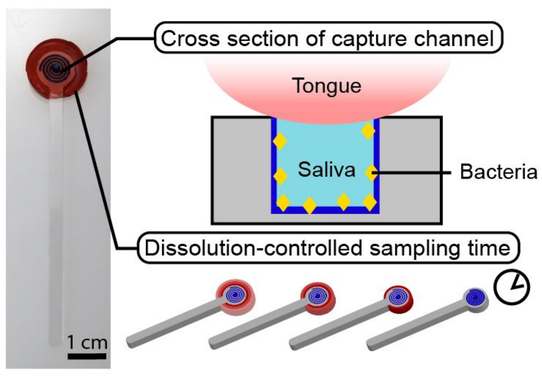
# Building a Large-Scale Nanopore Signal Classifier for the Human Proteome

Presenter: Hisham Bhatti, University of Washington Paul G. Allen Center for Computer Science and Engineering

Mentors: Melissa Queen and Jeff Nivala

## BACKGROUND

- The CandyCollect is an oral sampling device that utilizes open microfluidic channels to capture oral bacteria as a less invasive alternative to traditional sampling methods.
- Commensal bacteria S. aureus, S. mutans and S. pyogenes have been captured and eluted from the CandyCollect device[1].




Lee, et al. (2021)

## RESEARCH OBJECTIVES

1. **Detect respiratory pathogens using the CandyCollect in human subjects**
2. **Evaluate the efficacy of CandyCollect's ability to capture respiratory pathogens**

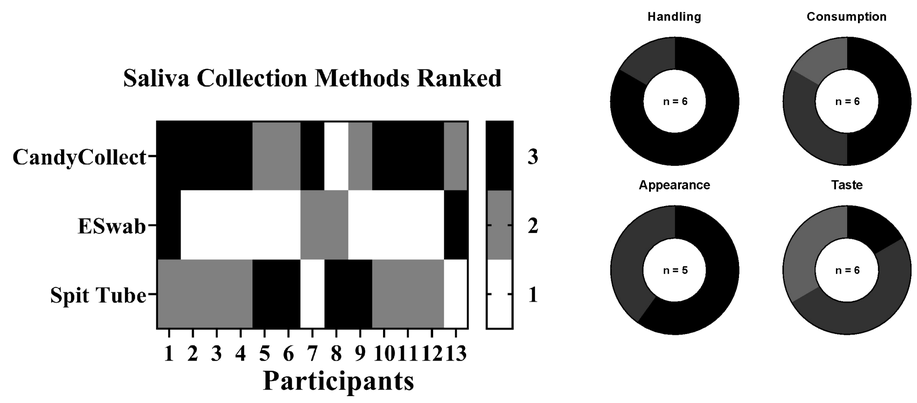
## REFERENCES

## HUMAN SUBJECTS

**UW IRB Approved STUDY00013842**


- Participants received test kits that included three saliva sampling devices (CandyCollect and two traditional devices)
- They then returned their samples for respiratory disease pathogen detection and provided their feedback on the different devices[2].

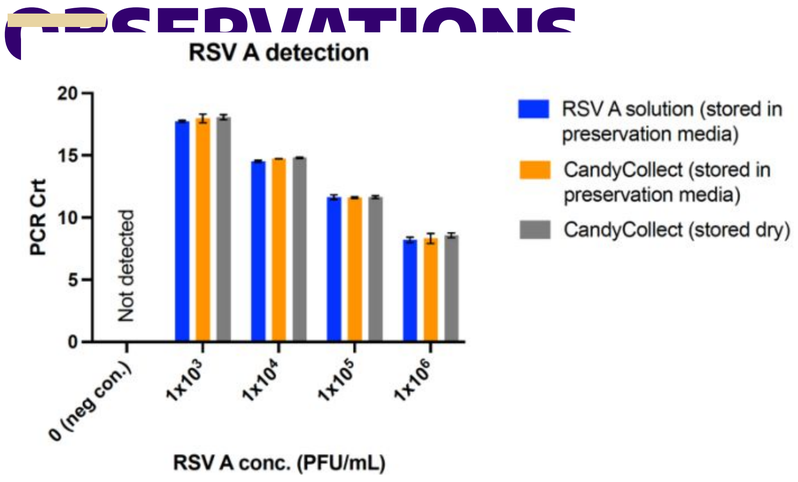
## User Feedback Data




## CONCLUSION

- Through quantitative Polymerase Chain Reaction (qPCR) we observed capture and elution pathogens of RSV A, S. pneumoniae, and respiratory syncytial virus (RSV A), and rhinovirus on the CandyCollect device.
- Participants reported (user feedback)

## DATA AND OBSERVATIONS




Initial in-lab experiments demonstrate that the amount of RSV A detected (orange and gray bars) is comparable to the amount applied (blue bar). Data are expressed as mean +/- SD (n=2).

| Detected pathogens | | | |
|---|---|---|---|
| Participant number | Oral swab | Nasal swab | CandyCollect |
| 1 | | RSV A | RSV A |
| 2 | | RSV A | RSV A |
| 3 | *S. pneumoniae* | *S. pneumoniae* | *S. pneumoniae* |
| 4 | Rhinoviruses | Rhinoviruses | Rhinoviruses |
| 5 | *S. pneumoniae* | *S. pneumoniae* | *S. pneumoniae* |

RSV A was detected in nasal swabs and the CandyCollect device (n=2). Streptococcus pneumoniae (n=2) and rhinovirus (n=1) were detected in all three sampling methods.

## FUTURE GOALS

- Test remaining samples with qPCR and analyze data
- Further streamline the CandyCollect device for younger users-parent child dyad study
- Improve efficiency in clinical settings.(how?) (mention candy as a timer?) (gummy)

**Acknowledgements:** N/A

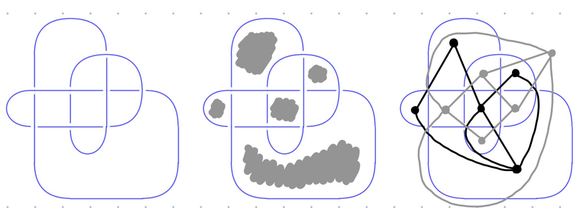
# An Algorithm for Implementing Tait Graph Functionality in SageMath

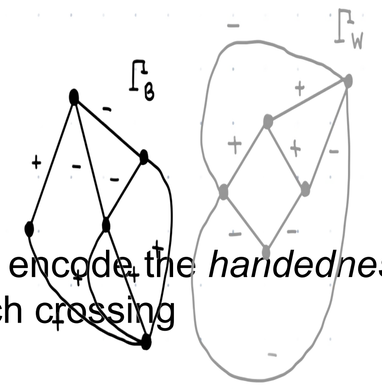
**Presenter: Hisham Bhatti, California State University: Chico Mathematics REU 2024**

**Mentor: Dr. John Lind**

## BACKGROUND

- A **knot** K is a smooth embedding of the circle $S_1$ onto $\mathbb{R}^3$
- We look at a **knot diagram** D, a projection of K onto $\mathbb{R}^2$
- The **Tait Graph** Γ is an undirected, +/- weighted graph associated to a knot diagram
  - Two disjoint Tait graphs: $\Gamma_B$ and $\Gamma_W$






- We encode the *handedness* of each crossing

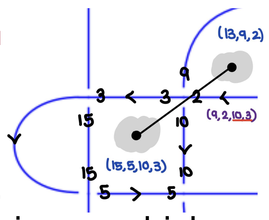

Left over Right

Right over Left

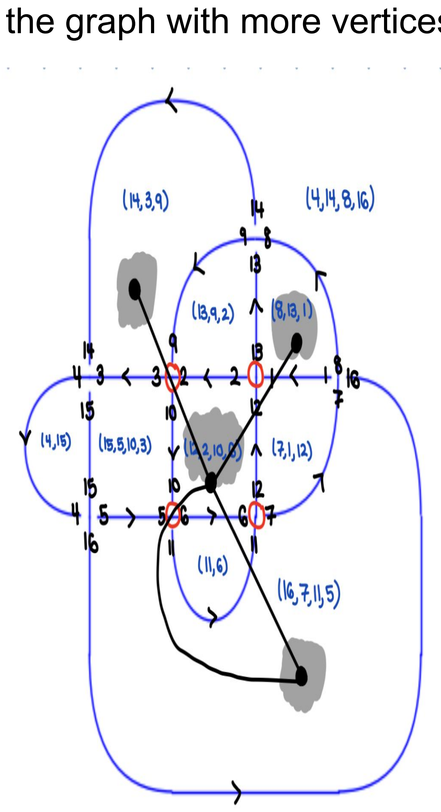
## RESEARCH OBJECTIVE

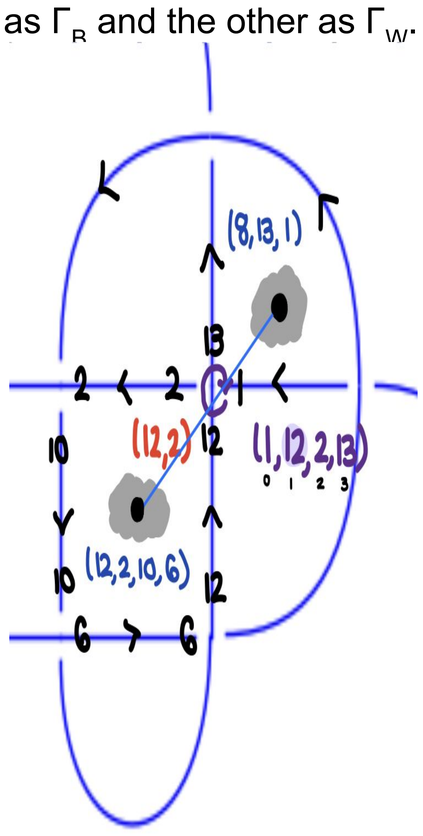
**Design an algorithm to construct a knot diagram's $\Gamma_B$ and $\Gamma_W$ Tait graph given its PD-code.**

## Regions Algorithm




1. Index the regions arbitrarily (1, 2, 3, …
2. Start with the first region $R_1$. Find crossings *c* which share two elements with the region. Such crossings are the *corners* of $R_1$.
1. Find the other *corner* in *c*. This corner is in a diagonal region $R_k$.
2. Continue this process starting at $R_2$ up to $R_n$, finding diagonal regions. Each region corresponds to a vertex, and crossings are edges between them.
3. Split the graph into its disconnected components. Return the graph with more vertices as $\Gamma_B$ and the other as $\Gamma_W$.



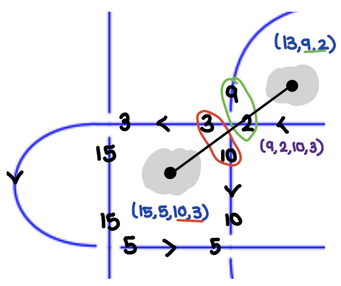




## Crossings Algorithm

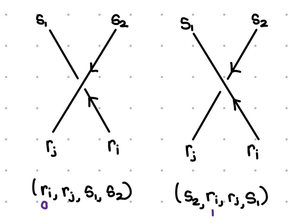
Given a region $R = (r_1, r_2, \ldots, r_k)$, a neighbor $S = (s_1, s_2, \ldots, s_l)$, and a crossing *C*

1. Decompose region *R* into cyclic pairs $[(r_1, r_2), (r_2, r_3), \ldots, (r_k, r_1)]$. Such pairs are the *corners* of *R*.
2. Find which pair $(r_i, r_j)$ matches the two strands at the crossing.

Cyclic Pairs for Region [15, 5, 10, 3]:




[15, 5] [5, 10] [**10, 3**] [3, 15]

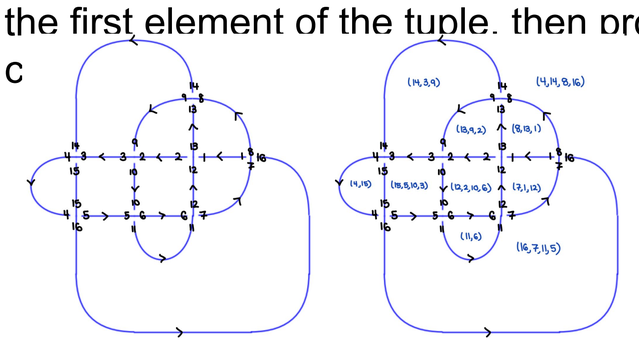



Note: A crossing in a PD code is the following: $C = (\underbrace{a}_{U}, \underbrace{b}_{O}, \underbrace{c}_{U}, \underbrace{d}_{O})$

3. Ind $C(r_i) = 0$ or 2 → left over right (+) Ind $C(r_i) = 1$ or 3 → right over left (-1)

## Planar Diagram (PD) Code

- Label an arbitrary incoming understrand with 1
- Follow along the knot with the orientation. When we hit a crossing, increment the label of the following strand by 1
- The incoming understrand will always be the first element of the tuple, then proceed c



(1, 12, 2, 13) (9, 2, 10, 3) (14, 3, 15, 4) (4, 15, 5, 16)
(10, 6, 11, 5) (6, 12, 7, 11) (16, 7, 1, 8) (13, 9, 14, 8)

## Regions

- Each region is defined by a tuple of the labels of the arcs enclosing it

## Results

- Tested our algorithm on 3000 knots up to 12 crossings (see KnotInfo)
  - Given a graph **Γ** = (V, E) with |V| = n, let $\mathbb{R}^{nxn}$,

$$D_{ii} = \sum_{e=(i,j)\in E} w_e \qquad A_{ij} = \sum_{e=(i,j)\in E} w_e$$

where $w_e$ = +1 if *e* is decorated with (+), $w_e$ = -1 if *e* is decorated with (-)

  - Define $\mathcal{L}$ = **D** - **A**. Construct the **Reduced Weighted Laplacian** $\mathcal{L}'$ by removing any row and column of $\mathcal{L}$
  - Should be that det($\mathcal{L}'$) = det(**G**), where **G** is the *Goertz Matrix*

See code at tinyurl.com/tait-graph

## Future Work

- Provably distinguish between $\Gamma_B$ and $\Gamma_W$
- Handle $R_1$ moves
- Calculate Jones Polynomial
- Implement Silver-Williams Directed Edges